

Instructions:

- We will only grade what is written in the space provided below each question
- Make sure to write your name and ERP on each page
- Extra sheets may be used for rough work but make sure to write your name and ERP on them. Answers on extra sheets will not be graded
- Understanding the questions is part of the exam; please do not ask for clarifications during the exam
- In this exam, 1k means 1000 and not 1024. Similarly, 1M means 10^6 . Capital 'B' is used for bytes, while small 'b' is used for bits. Therefore, 1kb is 1000 bits, and 4MB is 4×10^6 bytes
- Only writing or copying formulas without appropriate final numerical calculations will not earn you any credit.
- It is okay to leave the trivial steps and write the important ones. However, writing just the incorrect final answer will not earn you any partial credit.
- Make sure you have all 17 pages (printed on 9 two-sided sheets) of the exam before you begin

Name: _____

Solution

ERP: _____

Q1. Short Questions (3 marks)

a) OP_CHECKSIG (1 mark)

In Bitcoin, OP_CHECKSIG verifies a digital signature, even though a digital signature algorithm normally requires: the message, the public key, and the signature.

Why does the OP_CHECKSIG operator only take the public key and signature as inputs?

The message is implicitly derived from the transaction being validated. Bitcoin internally reconstructs the transaction digest (hash) according to the sighash rules, so the message does not need to be explicitly pushed onto the stack.

b) Digital Signatures Verification (1 mark)

What vulnerability may arise if a digital signature is verified directly against the message itself instead of verifying the hash of the message?

If verification is performed directly against the message, an attacker may generate a random garbage signature-like value Y and derive a corresponding garbage message M that still appears valid during verification.

Instead, digital signatures are verified against the cryptographic hash $H(M)$. Since the receiver independently computes the hash of the received message and compares it with the verified hash value, an attacker cannot feasibly create another message that produces the same hash.

c) Timelocks (1 mark)

Bitcoin interprets locktime value L as:

- a block height if $L < 500,000,000$
- and a Unix timestamp if $L \geq 500,000,000$

Why does this not conflict with real Bitcoin block heights? Also explain why the same "500 million rule" cannot be safely reused for relative timelocks.

Bitcoin block heights are far smaller than 500 million, so values above 500 million can safely be distinguished as Unix timestamps.

This rule cannot be reused for relative timelocks because relative timelocks represent durations rather than absolute global values. Therefore, the interpretation must be encoded explicitly instead of relying on a large numerical threshold.

Q2. Lightning Network Payment Channels (13 marks)

Alice and Bob open a Lightning Network payment channel for micropayments.

Initially, both users contribute funds into a 2-of-2 multisig funding transaction. During the lifetime of the channel, Alice and Bob exchange multiple off-chain commitment transactions that update their balances without broadcasting transactions on-chain.

Assume that:

- commitment transactions are asymmetric,
- revoked commitment states should not be reused,
- and delayed outputs are used for anti-cheating purposes.

a) Funding Transaction (2 marks)

Suppose Alice contributes 0.03 BTC and Bob contributes 0.05 BTC to open the payment channel.

The on-chain funding transaction fee is 800 sats, shared proportionally to their contributions.

Determine:

- i. The total value locked in the funding transaction – 1 mark
- ii. The fee paid by Alice – 0.5 marks
- iii. The fee paid by Bob – 0.5 marks

Step 1 – Total Value Locked

Total channel balance:

$$0.03 + 0.05 = 0.08 \text{ BTC}$$

Total locked value:

$$0.08 \text{ BTC} = 8,000,000 \text{ sats}$$

Step 2 – Fee Sharing

Total contribution:

$$0.03 + 0.05 = 0.08 \text{ BTC}$$

Alice's fee share:

$$800 \times \frac{0.03}{0.08} = 300 \text{ sats}$$

Bob's fee share:

$$800 \times \frac{0.05}{0.08} = 500 \text{ sats}$$

b) Commitment Transactions and Revocation Keys (3 marks)

Lightning Network commitment transactions are asymmetric and include revocation keys and timelocked outputs.

Answer the following:

- i. Why does each participant maintain their own commitment transaction? – 1 mark

Commitment transactions are asymmetric because each participant holds their own version that they can independently broadcast to settle the channel.

This allows either participant to close the channel unilaterally without requiring cooperation from the other party.

ii. Why are revocation keys exchanged whenever the channel state is updated? – 1 mark

Revocation keys invalidate old commitment states.

After a new channel state is agreed upon, both parties exchange revocation secrets for the previous state so that outdated commitment transactions can no longer be safely used.

iii. Why are some outputs timelocked in commitment transactions? – 1 mark

Timelocked outputs give the honest participant enough time to detect and respond if the other participant broadcasts an outdated revoked commitment transaction.

This delay is part of Lightning Network's anti-cheating mechanism.

c) Anti-Cheating Mechanism (3 marks)

Suppose Bob attempts to cheat by broadcasting an old, revoked commitment transaction that gives him a higher balance than the latest channel state.

Explain conceptually:

i. How Alice can detect this cheating attempt – 1 mark

Alice monitors the blockchain and can observe if Bob broadcasts an old revoked commitment transaction.

ii. Why the revocation mechanism prevents Bob from safely using old commitment states – 2 marks
Do not write Bitcoin scripts.

When the old state was revoked, Bob previously shared the revocation secret associated with that state.

Therefore, if Bob later broadcasts the revoked commitment transaction, Alice can use the revocation information to claim the funds before Bob's timelocked output becomes spendable.

Thus, broadcasting old states becomes financially dangerous for the cheating party.

d) Settlement Transaction (5 marks)

After several micropayments, the final channel balances are:

- Alice: 0.061 BTC
- Bob: 0.019 BTC

Bob broadcasts the final settlement transaction to close the channel.

Construct the high-level structure of the settlement transaction.

Clearly specify:

- the input,
- the outputs,
- which outputs are timelocked (if any),
- the normal spending path,
- the revocation spending path (used if the commitment state is revoked),
- and the witness/signatures required.

In the scriptPubKey, describe the spending conditions in words only (do not write Bitcoin scripts).

Inputs**Input 1**

- Previous TXID: Funding transaction TXID
- Output Index: Funding transaction multisig output
- scriptSig: Empty (SegWit-style transaction)

Outputs**Output 1 — Alice's Balance**

- Value: 0.061 BTC

Spending Condition

- Spendable immediately by Alice using her signature

Timelock

- No timelock

Revocation Path

- None required

Output 2 — Bob's Balance

- Value: 0.019 BTC

Normal Spending Path

- Spendable by Bob after a relative timelock expires

Revocation Path

- Alice may immediately spend this output using the revocation key if Bob broadcasts a revoked commitment transaction

Timelock

- Relative timelock applied before Bob can spend funds normally

Witness

Witness contains:

- Alice's signature
- Bob's signature
- The witness/redeem script corresponding to the funding multisig condition

(Conceptually this is commonly implemented using a P2WSH-based multisig construction in Lightning Network channels.)

Q3. Ethereum Gas and Transaction Nonce (10 marks)**a) Gas and Turing-Completeness (4 marks)**

Ethereum supports a Turing-complete programming language for smart contracts. While this allows developers to write powerful decentralized applications, it also introduces risks such as infinite loops and never-ending computations.

Explain how Ethereum's gas mechanism helps address the disadvantages of Turing-completeness.

In your answer, discuss:

- why gas is required,
- what happens when gas runs out during execution,
- and how gas helps protect the Ethereum network from misuse.

Ethereum uses a Turing-complete programming language, which means smart contracts can contain loops and complex computations. Without restrictions, this could allow infinite loops or extremely expensive computations that never terminate.

To solve this problem, Ethereum introduces a computational cost mechanism called gas.

Every computational operation in Ethereum has a predefined gas cost. When sending a transaction, users must provide a gas limit specifying the max amount of computation they are willing to pay for.

If contract execution consumes all available gas before completion:

- execution stops immediately,
- all state changes are reverted,
- but the gas already consumed is not refunded.

Unused gas is returned to the user after successful execution.

Gas helps Ethereum:

- limit computational resource usage,
- prevent denial-of-service attacks,
- discourage malicious infinite loops,
- and ensure that validators are compensated for computational effort.

Thus, gas allows Ethereum to safely support Turing-complete smart contracts.

b) Transaction Nonce Scenarios (6 marks)

Consider the following independent scenarios involving Ethereum transaction nonces.

For each scenario:

- i. State whether the transaction(s) will be accepted or rejected by the network
- ii. Explain what will happen
- iii. Identify the purpose or advantage of the nonce behavior in that scenario

Scenario 1 – 2 marks

At time t , Alice's account nonce in the Ethereum world state is 5.

Alice now broadcasts a new transaction with nonce 4.

The transaction will be rejected by the network.

Alice's current account nonce is already 5, meaning the network has previously processed transactions with nonces 0–4.

A new transaction with nonce 4 is considered outdated and invalid.

Purpose of Nonce

This prevents replay attacks by ensuring old transactions cannot be processed again.

Scenario 2 – 2 marks

Alice currently has one pending unconfirmed transaction with nonce 5 and a low transaction fee. She then creates another transaction with nonce 6 and offers a much higher transaction fee.

Both transactions are valid and may remain pending in the mempool.

The network must process the nonce 5 transaction before processing the nonce 6 transaction because Ethereum transactions from the same account must execute in nonce order.

Validators may prioritize the higher-fee nonce 6 transaction indirectly by first including the earlier pending transaction.

Purpose of Nonce

Nonce ordering ensures transactions execute in the intended sequence.

This technique can also encourage faster confirmation of an older stuck transaction.

Scenario 3 – 2 marks

Alice has a pending unconfirmed transaction with nonce 5.

She creates a second transaction with the same nonce 5 but with a much higher transaction fee.

Both transactions initially appear valid because they have the same nonce.

However, once one transaction is confirmed, the other becomes invalid because the account nonce increases.

Validators will most likely choose the transaction with the higher transaction fee.

Purpose of Nonce

This mechanism allows users to replace or cancel a pending transaction by submitting another transaction with the same nonce and a higher fee.

Q4. Ethereum State Trie (Modified Merkle Patricia Trie) (10 marks)**Ethereum MPT Node Types**

Ethereum defines three types of nodes in its Modified Merkle Patricia Trie (MPT):

- **Leaf Node:** A node with no child nodes.
- **Extension Node:** A node with exactly one child node.
- **Branch Node:** A node with more than one child node (between 2 and 16 children).

This distinction allows Ethereum to compress the trie and reduce unnecessary storage.

Node Contents

- **Leaf Node:** Stores the remaining key path and the final value (e.g., the account state or account balance).
- **Extension Node:** Stores a shared key path that is common among multiple accounts before the trie branches again.
- **Branch Node:** Contains 17 fields
 - 16 slots corresponding to hexadecimal digits 0–f
 - 1 optional value field

The next nibble of the key determines which branch slot should be followed.

Consider the following simplified Ethereum world state, where each account address is represented using a **7-nibble hexadecimal string**.

Simplified World State	
Account Keys	Value
5 1 a 3 4 7 c	2 ETH
5 1 a 3 8 9 d	5 ETH
5 2 b 7 1 1 a	1 ETH
5 2 b 9 f 2 c	3 ETH

Construct the Ethereum Modified Merkle Patricia Trie (MPT) for these accounts.

Represent nodes using Ethereum's standard node types:

- Branch nodes
- Extension nodes
- Leaf nodes

Use the following prefixes where required:

- 0x0 → Extension node, even-length path
- 0x1 → Extension node, odd-length path
- 0x2 → Leaf node, even-length path
- 0x3 → Leaf node, odd-length path

Clearly show shared prefixes, branch points, and remaining nibbles stored in each node.

Available Building Blocks:

The following figures illustrate the standard Ethereum Modified Merkle Patricia Trie (MPT) node types that may be used while constructing the trie in this question.

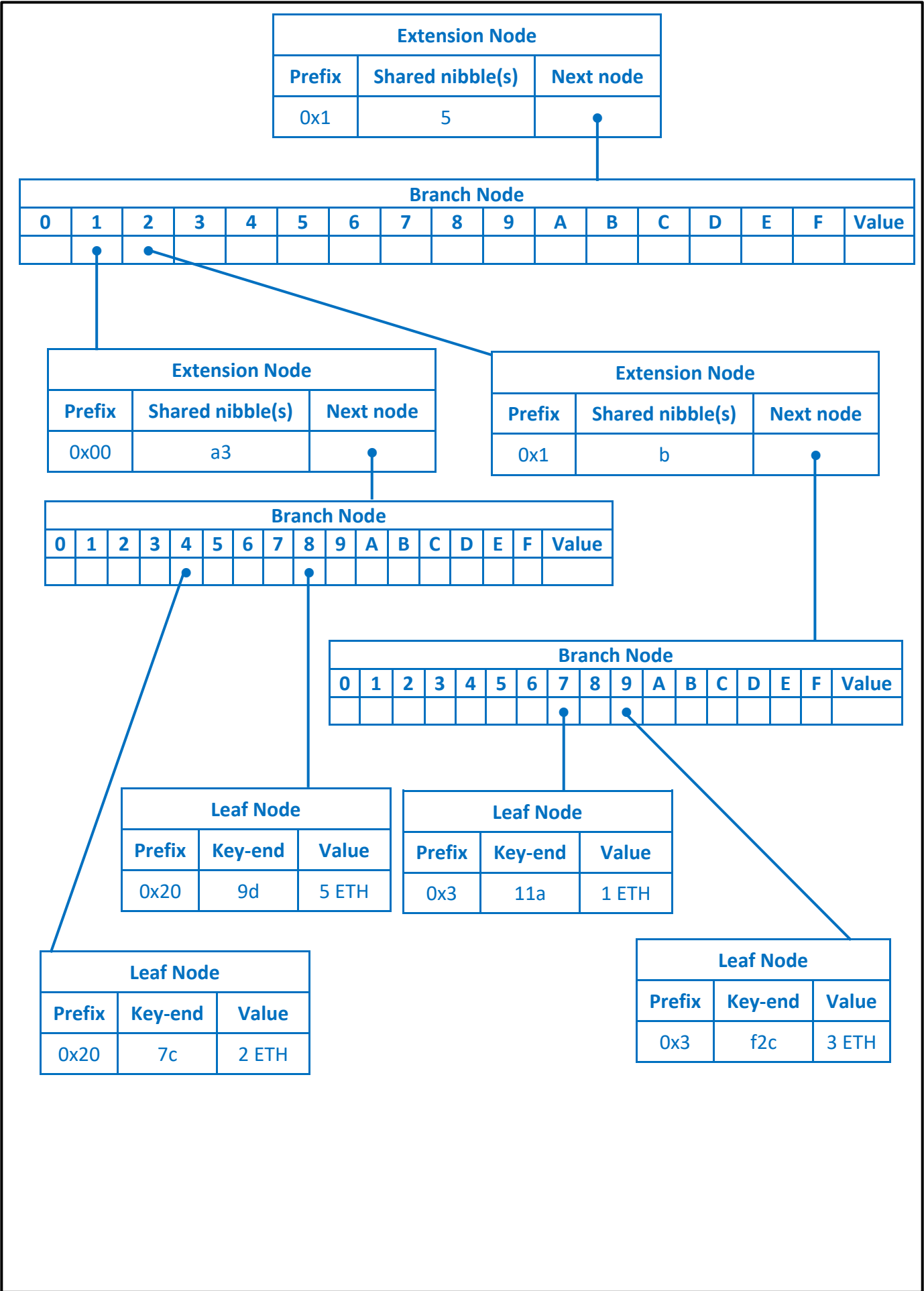
Extension Node		
Prefix	Shared nibble(s)	Next node

Leaf Node		
Prefix	Key-end	Value

Branch Node																
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Value

You may use these building blocks as references while drawing your final trie structure.

Name/Erp: _____



Q5. Ethereum: Proof of Work vs Proof of Stake (10 Marks)

Ethereum originally used **Proof of Work (PoW)** but later transitioned to **Proof of Stake (PoS)**.

Answer the following:

a) Explain how block validation works in: (4 marks)

i. Proof of Work – 2 marks

Proof of Work

In Proof of Work, transactions are validated by **miners**. Miners compete to solve complex mathematical puzzles using computational power. The first miner to solve the puzzle adds the next block to the blockchain and receives a reward.

ii. Proof of Stake – 2 marks

Proof of Stake

In Proof of Stake, transactions are validated by **validators**. Validators lock (stake) cryptocurrency in the network, and validators are selected to create/validate blocks based on their stake.

b) Compare PoW and PoS in terms of: (4 marks)

- resource requirement, – 1 mark
- miner/validator selection mechanism, – 1 mark
- probability of miner/validator being selected to produce a block, – 1 mark
- and security against a 51% attack – 1 mark

Aspect	Proof of Work	Proof of Stake
Resource requirement	Requires large computational power and high electricity consumption	Requires staking cryptocurrency with much lower energy usage
Miner/validator selection mechanism	Miners compete to solve cryptographic puzzles	Validators are selected from participants who stake cryptocurrency
Probability of being selected	Depends on computational/hash power	Depends on the amount of cryptocurrency staked
51% attack	Attacker must control more than 51% of the network's computational power	Attacker must own/control more than 51% of the staked cryptocurrency

c) Briefly explain one major reason Ethereum switched from PoW to PoS. (2 marks)

Ethereum switched from PoW to PoS mainly because PoS is much more energy efficient and scalable. It reduces electricity consumption and supports Ethereum's long-term scaling goals.

Q6. Comparing Stablecoin Stability Mechanisms (12 marks)

Stablecoins use different mechanisms to maintain price stability depending on their design.

Consider the following three categories of stablecoins:

- Fiat Currency / Non-Currency Asset-Backed Stablecoins
- Cryptocurrency-Backed Stablecoins
- Algorithmic Non-Collateralized Stablecoins

Answer the following questions:

a) Explain how fiat currency-backed or non-currency asset-backed stablecoins maintain their peg through arbitrage. (4 marks)

Fiat currency-backed or non-currency asset-backed stablecoins maintain stability by allowing users to redeem the stablecoin for the underlying reserve asset at a fixed pegged value.

If the market price deviates from the peg, arbitrage traders buy or redeem the stablecoin to profit from the price difference. This market activity pushes the price back toward the peg.

b) Explain how cryptocurrency-backed stablecoins maintain stability. In your answer, discuss: (4 marks)

- over-collateralization – 2 marks
- automatic liquidation – 2 marks

Cryptocurrency-backed stablecoins maintain stability through over-collateralization. Users must lock cryptocurrency collateral worth more than the value of the stablecoins they issue.

For example, a user may lock \$150 worth of ETH to mint \$100 worth of stablecoins. The extra collateral acts as a safety buffer against cryptocurrency price volatility.

If the collateral value falls below a required threshold, the protocol automatically liquidates part of the collateral. The collateral is sold to repay the outstanding stablecoin debt and protect the peg.

c) Explain how algorithmic non-collateralized stablecoins maintain their peg when: (4 marks)

- the stablecoin price rises above the peg – 2 marks
- the stablecoin price falls below the peg – 2 marks

Algorithmic non-collateralized stablecoins maintain stability by dynamically adjusting the token supply.

When the stablecoin price rises above the peg, the protocol increases supply by minting additional stablecoins. Companion token holders may be rewarded as part of this expansion process. Increasing supply helps push the price back down toward the peg.

When the stablecoin price falls below the peg, the protocol attempts to reduce supply by incentivizing users to burn stablecoins. This may involve offering discounted companion tokens in exchange for burning stablecoins. Reducing supply is intended to push the price back upward toward the peg.

Q7. Private Institutional Stablecoins (12 marks)

Consider the following scenario:

Company A needs to pay Company B \$3 million for goods.

Both companies have accounts with the same large bank.

Using the traditional banking system:

- the transfer relies on systems such as SWIFT or Fedwire
- settlement may take several hours or even a full business day
- intermediaries and manual reconciliation increase operational complexity and cost

Suppose the bank instead introduces a private institutional stablecoin that is:

- issued and managed by the bank itself
- fully backed by fiat reserves held by the institution
- usable only by the bank's customers on the bank's private blockchain network

Answer the following questions:

a) Explain how the payment process could work using the bank's private institutional stablecoin instead of the traditional banking system. (4 marks)

Company A would first convert \$3 million from its bank account into the bank's stablecoin. The bank would issue an equivalent amount of digital tokens on its private blockchain.

Company A would then transfer the stablecoins directly to Company B on the bank's internal blockchain network. Since both companies are customers of the same institution, the transfer can settle immediately within the bank's system.

Company B could then redeem the received stablecoins back into USD through the bank.

b) Explain how this approach improves settlement efficiency as compared to the traditional banking system. (3 marks)

The private institutional stablecoin improves settlement efficiency because transactions occur directly on the bank's private blockchain without relying on external intermediaries such as SWIFT.

Settlement can occur within seconds and may operate 24/7 instead of depending on banking hours.

The system also reduces reconciliation overhead and operational costs because all transfers are recorded on a shared internal ledger.

c) State TWO characteristics of private institutional stablecoins that distinguish them from public stablecoins such as USDC or DAI. (3 marks)

Possible characteristics include:

- Operates only within a closed private ecosystem
- Accessible only to the institution's customers
- Not publicly traded on cryptocurrency markets
- Fully controlled by the issuing institution
- Supply and redemption are centrally managed

(Any two)

d) Explain why such a system is typically described as centralized. (2 marks)

The system is centralized because a single institution controls:

- issuance of the stablecoin
- redemption of tokens
- user access
- reserve management
- operation of the blockchain network

Users must trust the institution rather than a decentralized public network.

Q8. Smart Contract Security in IBA CafeteriaPay (12 marks)**Scenario**

IBA Karachi launches an experimental blockchain-based payment system called **IBA CafeteriaPay**.

Students can:

- deposit ETH into their cafeteria account,
- withdraw part of their balance,
- or close their account and withdraw all remaining funds.

The following Solidity-like smart contract is used:

<pre>contract IBACafeteriaPay { mapping(address => uint8) public balance; function deposit() public payable { balance[msg.sender] += uint8(msg.value); } function withdraw(uint256 amount) public { require(balance[msg.sender] >= amount); (bool success,) = msg.sender.call{value: amount}(""); require(success); } }</pre>	<pre> balance[msg.sender] -= uint8(amount); } function closeAccount() public { uint256 amount = balance[msg.sender]; (bool success,) = msg.sender.call{value: amount}(""); require(success); balance[msg.sender] = 0; } }</pre>
---	---

Additional Note:

- In Ethereum, smart contracts internally store ETH amounts in wei, where:
1 ETH = 10^{18} wei
- The IBA cafeteria system is expected to support ordinary purchases such as tea (~500 wei), drinks (~2,000 wei), burgers (~15,000 wei), sandwiches (~8,000 wei), and full meal deals (~25,000 wei)

A security audit reports that the contract contains vulnerabilities.

a) Identify the Vulnerabilities (6 marks)

Identify **two vulnerabilities** present in the contract.

For each vulnerability:

- Name the vulnerability – 1 mark
- Specify the affected function(s) – 1 mark
- Briefly explain the security risk – 1 mark

(2 vulnerabilities × 3 marks)

Vulnerability 1: Reentrancy Vulnerability

- **Affected Functions:** withdraw() and closeAccount()
- **Explanation:**

Both functions send ETH to msg.sender using the low-level call() function before updating the user's balance.

Since call() transfers control to the recipient contract, a malicious student contract can repeatedly call withdraw() or closeAccount() before the balance is updated.

This may allow the attacker to withdraw funds multiple times and drain the contract.

Vulnerability 2: Integer Overflow Vulnerability

- **Affected Function:** deposit()
- **Explanation:**

The contract stores balances using uint8, whose maximum value is 255.

If a student deposits more than 255 wei, the balance can overflow and wrap around to a smaller incorrect value.

For example:

$$255 + 1 = 0 \text{ (overflow in uint8)}$$

This causes incorrect balance accounting.

b) Propose Fixes (6 marks)

For each vulnerability identified in part (a):

1. Explain how the contract should be modified to fix the issue – 2 marks
2. Briefly explain why your fix works – 1 mark

(2 vulnerabilities × 3 marks)

Fix for Reentrancy Vulnerability**Modified Approach**

Update the user's balance before sending ETH.

Example:

```
function withdraw(uint256 amount) public {  
  
    require(balance[msg.sender] >= amount);  
  
    balance[msg.sender] -= uint8(amount);  
  
    (bool success, ) = msg.sender.call{value: amount}("");  
    require(success);  
}
```

Similarly, in closeAccount():

```
function closeAccount() public {  
  
    uint256 amount = balance[msg.sender];  
  
    balance[msg.sender] = 0;  
  
    (bool success, ) = msg.sender.call{value: amount}("");  
    require(success);  
}
```

Why This Fix Works

The balance is updated before the external call occurs.

Therefore, if the attacker attempts a reentrant call, the balance has already been reduced (or set to zero), so the attack fails.

Fix for Integer Overflow Vulnerability**Modified Approach**

Replace:

mapping(address => uint8) public balance;

with:

mapping(address => uint256) public balance;

Why This Fix Works

uint256 can store very large values and matches the type used by msg.value.

This prevents overflow during deposits and ensures balances are stored correctly.

Q9. ERC-20 Token Limitations and ERC-777 Improvements (18 marks)

Consider the following simplified staking contract:

```
contract SimpleStaking {
```

```
    IERC20 public stakingToken;
```

```
    // stores amount staked by each user
```

```
    mapping(address => uint256) public stakedBalance;
```

```
    // custom event to be emitted whenever a user successfully stakes tokens
```

```
    event Deposited(address indexed user, uint256 amount);
```

```
    constructor(address tokenAddress) {
```

```
        stakingToken = IERC20(tokenAddress);
```

```
    }
```

```
    // user deposits tokens into staking contract
```

```
    function deposit(uint256 amount) external {
```

```
        // transfer tokens from user into contract
```

```
        stakingToken.transferFrom(msg.sender, address(this), amount);
```

```
        // update user stake balance
```

```
        stakedBalance[msg.sender] += amount;
```

```
        // emit deposit event
```

```
        emit Deposited(msg.sender, amount);
```

```
}  
  
// returns stake balance of a user  
function getStake(address user) external view returns(uint256) {  
    return stakedBalance[user];  
}  
}
```

Assume stakingToken follows the ERC-20 standard.

A user transfers 100 tokens to the SimpleStaking contract's address:

```
stakingToken.transfer(SimpleStakingAddress, 100);
```

After the transfer:

- the staking contract successfully receives the 100 tokens
- but calling *getStake(user)* still returns 0

Answer the following questions:

a) Explain why the user's stake balance was not updated even though the staking contract received the tokens. (3 marks)

The user directly transferred tokens to the staking contract using ERC-20 *transfer()*.

In ERC-20, transferring tokens only changes token balances inside the token contract. The receiving contract is not automatically notified that tokens arrived.

Therefore, the *deposit()* function in SimpleStaking was never executed. Since the logic that updates *stakedBalance[user]* exists inside *deposit()*, the staking balance remained unchanged even though the contract received the tokens.

b) Explain the purpose of *approve()* in ERC-20 and why the *deposit()* function depends on it. (3 marks)

approve() gives another address or smart contract permission to spend tokens on behalf of the user.

In the *deposit()* function, the staking contract calls *transferFrom()* to move tokens from the user into the contract. This operation can only succeed if the user previously authorized the staking contract using *approve()*.

Without approval, the contract is not allowed to transfer the user's ERC-20 tokens.

c) ERC-777 was designed to address this ERC-20 limitation. (5 marks)

Explain what mechanism ERC-777 introduces and how it changes the way smart contracts can process incoming tokens.

ERC-777 introduces a hook/callback mechanism that allows receiving contracts to automatically react when tokens arrive.

When tokens are sent to a smart contract, a function such as *tokensReceived()* is automatically triggered on the receiving contract.

This changes the interaction model because contracts no longer receive tokens silently. The receiving contract can immediately process deposits, update balances, emit events, validate transfers, or reject tokens.

d) Suppose the staking system is redesigned using ERC-777 tokens instead of ERC-20 tokens. (7 marks)

Explain how to implement the logic to correctly handle the user's stake in this scenario.

Write pseudocode for your solution. – 5 marks

Also explain why:

- the explicit *approve()* + *transferFrom()* pattern is no longer necessary – 1 mark
- a separate *deposit()* call may no longer be required – 1 mark

Example pseudocode:

```
function tokensReceived(operator, from, to, amount) {  
  
    stakedBalance[from] += amount;  
  
    emit Deposited(from, amount);  
}
```

In ERC-777, users can directly send tokens to the staking contract using *send()*.

The receiving contract is automatically notified through the hook mechanism, so the staking logic executes immediately when tokens arrive.

Because of this:

- the contract no longer needs to pull tokens using *transferFrom()*
- prior approval using *approve()* is unnecessary
- a separate *deposit()* function may also become unnecessary because token transfer and stake processing occur in a single interaction